Week 45, Tuesday Session

Flipped classroom: your questions, then Project 2 only – the regression half against Project 1, the classification half against logistic regression, and how to write it up

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

How today works

The format (as every Tuesday)

Mondays are lectures; Tuesdays belong to your questions and to concrete cases. Today the cases are *your own results*: the project you hand in on Monday November 9.

1. **First ∼20 min**: your open questions from Monday's lecture – the hidden state and the unrolled graph, backpropagation through time, the product of Jacobians and the memory horizon, the LSTM gates and the forget-gate bias, the adding problem – discussed in plenum.
2. **Rest of the double session**: *two* cases, worked in your project groups with your Project 2 code, figures and draft open. Case 1: the regression half (parts b–d) – your network against your own OLS, Ridge and Lasso from Project 1, and the hyperparameter maps. Case 2: the classification half and the evaluation (parts e–f) – MNIST against the logistic-regression baseline, the confusion matrix, and the critical comparison. Each case ends with the questions a grader – or an examiner – will ask you.
3. **Last 5 min**: wrap-up, how to hand in, and what Monday builds on.

No weekly exercises this week

The week is for finishing the project. **Project 2 is due Monday November 9.**

Picking up from Monday – and where you should be with Project 2

Monday in one sentence each

A CNN shares parameters in space, a recurrent network in time: it carries a state forward and reuses U, W, V at every step – a learned time-stepper, Eq. (11.13); in Keras and PyTorch one layer fed a `(samples, timesteps, features)` tensor, in the same `Sequential`/training-loop pattern as the CNN. Unrolled, it is a deep network with tied weights, and `loss.backward()` is BPTT, Eqs. (11.15)–(11.18). The gradient across a lag is a product of Jacobians (Theorem 11.1): 10^{-18} over 59 steps at $\rho(W) = 0.5$. Clipping fixes explosion only. The simple RNN learned the oscillator's rule and fails on the adding problem from $T = 30$; gated cells next Monday.

Questions we expect

Why is the second term of Eq. (11.16) absent in the sequence-to-label case? Why tanh and not ReLU? Why did the network trained on one trajectory predict a different one? Why does PyTorch print 673 parameters where Keras prints 649? Ask.

Project 2: what should be done by now

Part	Done in	Status
a) costs, activations, derivatives	wk 40–41	written up
b) own network on Runge, vs OLS	wk 41–42	figures final
c) activations, depth, width	wk 42	heat maps
d) L_2/L_1 vs Ridge/Lasso	wk 42	table
e) MNIST, baseline, confusion matrix	wk 43	**Case 2 today**
cross-validated hyperparameters	wk 44	**Case 1 today**
f) evaluation, abstract, conclusion	now	drafted

What remains

The *analysis* – the six items of "Required analysis" in the project text – and the text around it; a clean re-run from a fresh kernel so that the numbers in the report are the numbers the code produces; the LLM appendix; the GitHub link. Today is about the analysis.

The analysis is where the points are: what a figure must do

A figure states a finding

- One claim per figure, in the caption, with the number in it: "The network reaches a test MSE of 0.0041 ± 0.0006 at $\gamma = 10^{-2}$, $\lambda = 10^{-4}$, against 0.0039 for OLS at degree 8."
- Logarithmic axes where the quantity spans decades (γ, λ, MSE).
- Heat maps for two hyperparameters at a time (project text, analysis item 2): colour = cross-validated score, the chosen cell marked, the range wide enough to show *bad* performance on both sides.
- Error bars from the folds (week 44) wherever a single number is compared with another single number.

A table compares on equal terms

- Same data, same split, same scaling, same seed for every row.
- Hyperparameters chosen on training data only (validation split or k-fold, Section 3.14); the test number computed *once* and choosing nothing.
- Parameter counts next to the scores: a 1–50–1 network has 151 parameters, a degree-8 polynomial 9.
- Say what "best" means: lowest CV MSE, or simplest within one standard error (Section 3.14)?

The sentence after the figure: its "so what" – what moved, why (which section of the notes), and what you decided because of it.

Case 1: the regression half (parts b–d): your network against Project 1

Goal: leave with the two figures and the one table that carry the regression argument of your report, each with a caption that states a finding, and with the paragraph that answers analysis items 1–3 drafted.

1. **Assemble the evidence** (your own code, the Runge data of Project 1): (A) training and test MSE against epochs for the 1–50–1 and 1–100–100–1 networks with plain GD, RMSprop and Adam at the learning rates you scanned (part b); (B) the heat map of CV MSE over (γ, λ) for your best architecture, L_2 and L_1 separately (part d); (C) the heat map over (hidden nodes, activation) or (layers, nodes) (part c).
2. **The table:** OLS, Ridge and Lasso at their Project-1 best (degree, λ) against the network at its cross-validated best, with CV MSE $\pm$ standard error and the *test* MSE computed once – plus the parameter count of each row.
3. **Cross-examine:** does the network beat OLS on a smooth one-dimensional function? (It often does not, and that is a result, not a failure.) Does L_2 on the weights behave like Ridge – a shallower, later minimum – and L_1 like Lasso? Where does the sigmoid network stall and the ReLU one not, and what does Section 8.9 say about it?
4. **Draft the paragraph:** which model, why, with the bias-variance language of Chapter 2 and the parameter counts.

Case 1: what you should find – and what it looks like when it is wrong

The picture that should emerge (qualitatively)

(A) Plain GD at a fixed γ is slow and either rattles or creeps; Adam reaches the plateau in a fraction of the epochs and is insensitive to γ over a decade (week 38); RMSprop sits between. The 1–50–1 ReLU network reaches a test MSE comparable with the best OLS polynomial of Project 1 – often slightly worse, with fifteen times the parameters. The deeper network is not better on this problem and overfits sooner. (B) The L_2 heat map has a valley: too small a λ changes nothing, too large one underfits (the Ridge shrinkage of Eq. 3.48 in a different disguise); the L_1 valley is narrower and the network loses nodes as λ grows. (C) Sigmoid hidden layers train slowest and stall first as the depth grows (Section 8.9: $\sigma' \leq 1/4$ per layer); ReLU and leaky ReLU are close; width beyond ~ 50 buys little on $n = 100$ points.

Symptoms, and what they mean

- Test MSE far below $\sigma^2 = 0.01$: you scaled with the whole data set before splitting, or evaluated on training data (Section 3.13).
- Network MSE constant in γ over four decades: Adam is doing its job – say so – or the run never left the initialisation; check the training curve.
- ReLU network at exactly the mean of y: dead units from a large γ or a bad initialisation; use He initialisation (Eq. 8.74) and halve γ.
- L_1 and L_2 results identical: the penalty is in the cost but not in the gradient, or λ never reached the scale of the weights.
- Different seed, different "best" architecture: the minimum is flat; report the region, the error bar and the simplest model inside it.
- Network beats OLS by a factor ten: check that OLS was given its Project-1 best degree and the same scaled data.

Case 1: do you have the insight?

1. Your 1–50–1 network has a cross-validated MSE of 0.0112 ± 0.0009; OLS at degree 8 has 0.0105 ± 0.0008. Which model do you recommend for this problem, and what do you write in the conclusion about neural networks for regression?
2. Adam at $\gamma = 10^{-3}$ and at $\gamma = 10^{-2}$ give the same final MSE; plain gradient descent needs $\gamma = 0.5$ and 5000 epochs to get there. Explain both observations in one sentence each.
3. With L_2 regularisation the heat map's valley sits at $\lambda \approx 10^{-4}$; Ridge in Project 1 wanted $\lambda \approx 10^{-2}$. Are the two λ's comparable at all?
4. The sigmoid network with three hidden layers stays at the mean of y for 2000 epochs and then suddenly learns. What happened, and which figure of the lecture notes would you cite?

Case 1: do you have the insight?

1. Your 1–50–1 network has a cross-validated MSE of 0.0112 ± 0.0009; OLS at degree 8 has 0.0105 ± 0.0008. Which model do you recommend for this problem, and what do you write in the conclusion about neural networks for regression?
2. Adam at $\gamma = 10^{-3}$ and at $\gamma = 10^{-2}$ give the same final MSE; plain gradient descent needs $\gamma = 0.5$ and 5000 epochs to get there. Explain both observations in one sentence each.
3. With L_2 regularisation the heat map's valley sits at $\lambda \approx 10^{-4}$; Ridge in Project 1 wanted $\lambda \approx 10^{-2}$. Are the two λ's comparable at all?
4. The sigmoid network with three hidden layers stays at the mean of y for 2000 epochs and then suddenly learns. What happened, and which figure of the lecture notes would you cite?

What we hope you say

(1) The two are within one standard error of each other; the simpler model – nine parameters, a closed-form solution, no learning rate – wins on every count except flexibility. The honest conclusion is that on a smooth one-dimensional function with 100 points a polynomial is a hard benchmark, and that the network's advantage is that it did not need the basis to be chosen (Chapter 8, the "learned features" insight of the project text). (2) Adam divides each component of the gradient by a running scale, so the step is of order γ in every direction and the result is insensitive to γ over a decade (Section 4.11); plain GD must use a single γ below $2/\lambda_{\max}$ for the stiffest direction and then crawls along the flat ones (Section 4.5). (3) No: Ridge penalises the polynomial coefficients, L_2 penalises weights and biases of a network whose scale is set by the initialisation, and the data are scaled differently; what *is* comparable is the *shape* of the MSE-against-λ curve and the qualitative effect – a later, shallower minimum. Compare mechanisms, not numbers. (4) Vanishing gradients: three sigmoid layers multiply the error by $\sigma' \leq 1/4$ three times, so the first layer barely moves until the weights have grown enough to carry a gradient (Section 8.9, Figure 8.14); ReLU or He initialisation removes the plateau. Cite the figure and your own training curve side by side.

Case 2: the classification half and the evaluation (parts e–f)

Goal: the MNIST table, the confusion matrix with its reading, the baseline comparison, and the critical paragraph of your conclusion drafted.

1. **Assemble the evidence** (full MNIST, the 60 000/10 000 split as intended): (A) test accuracy of logistic regression (your network with zero hidden layers, softmax + cross entropy from part a) – the required baseline; (B) test accuracy of your best network, its hyperparameters chosen on training data only (validation split or k-fold on a subset); (C) the 10×10 confusion matrix of (B) on the test set, counts and row-normalised (week 43).
2. **The table:** baseline, your network, and – if you did it – Scikit-Learn's `LogisticRegression` or `MLPClassifier` on the same split, with parameter counts and training time.
3. **Read the confusion matrix:** which digits are confused with which, and in which direction? Per-class precision and recall from its rows and columns (Section 5.12); which class would you flag to a user?
4. **Cross-examine the baseline:** *does* a hidden layer help, by how many points, and is that difference larger than the run-to-run spread (two or three seeds)? Analysis item 4 asks you to say *why*.
5. **Draft part f):** regression and classification side by side – where the network earned its parameters and where it did not; the gradient methods and learning rates (analysis item 6) in one paragraph with the evidence of Case 1.

Case 2: what you should find

The numbers that should emerge

Logistic regression on 784 scaled pixels: about 92% test accuracy (the baseline named in the project text). One hidden layer of 100 ReLU nodes with softmax and cross entropy, Adam, a few epochs: 97–98% – five to six points above the baseline, and the gap is far larger than the spread over seeds (± 0.2 points). Two hidden layers: a few tenths of a point more at most. For reference, a CNN reaches 99% (week 44) and an LSTM reading rows reaches 97% (Monday). The confusion matrix is dominated by its diagonal; the off-diagonal mass sits in a few pairs – $4 \leftrightarrow 9$, $3 \leftrightarrow 5$, $7 \leftrightarrow 9$, $3 \leftrightarrow 8$ – and is *not* symmetric: more 9s are read as 4 than the reverse, say, and that direction is the sentence to write. Precision and recall per class are within a few points of each other except for the confused pairs.

Symptoms, and what they mean

- Accuracy stuck at 11.35%: the network predicts one class – a dead ReLU layer or a γ too large for the cross entropy; check that $\delta^L = a - y$ is what your code computes.
- Training accuracy 100%, test 96%: overfitting; early stopping on a validation split, or L_2, or fewer epochs.
- Baseline at 98%: you trained logistic regression on the test set, or the "baseline" has a hidden layer.
- Confusion matrix with the classes in the wrong order, or rows summing to 10 000: check `labels=` and normalise by row for recall, by column for precision.
- Test accuracy quoted to four decimals: with 10 000 test images the binomial standard error at 97% is 0.17 points – two decimals, and say so.
- Hyperparameters tuned by looking at the test accuracy: the test number then chooses something and is no longer a test number. Re-split.

Case 2: do you have the insight?

1. Your network reaches 97.6% and logistic regression 92.1%. Why does a hidden layer help *here* when it did not help in the regression half?
2. Your confusion matrix shows 4 read as 9 in 31 cases and 9 read as 4 in 12. Write the sentence for the report, and say which of precision and recall of class 4 is affected by each number.
3. A fellow student reports 98.9% with a dense network trained for 200 epochs on the full training set, hyperparameters chosen by watching the test accuracy. What is wrong, and how large is the problem?
4. Analysis item 6 asks about gradient methods and learning rates. You have the Runge curves of Case 1 and three MNIST runs with Adam at $\gamma = 10^{-3}$. Is that enough evidence, and what would you add with one more hour of compute?

Case 2: do you have the insight?

1. Your network reaches 97.6% and logistic regression 92.1%. Why does a hidden layer help *here* when it did not help in the regression half?
2. Your confusion matrix shows 4 read as 9 in 31 cases and 9 read as 4 in 12. Write the sentence for the report, and say which of precision and recall of class 4 is affected by each number.
3. A fellow student reports 98.9% with a dense network trained for 200 epochs on the full training set, hyperparameters chosen by watching the test accuracy. What is wrong, and how large is the problem?
4. Analysis item 6 asks about gradient methods and learning rates. You have the Runge curves of Case 1 and three MNIST runs with Adam at $\gamma = 10^{-3}$. Is that enough evidence, and what would you add with one more hour of compute?

What we hope you say

(1) Logistic regression is a linear classifier in pixel space: it draws ten hyperplanes through 784 dimensions, and digits are not linearly separable there – a 4 and a 9 differ by a stroke that no single weighted sum of pixels isolates. The hidden layer learns features (strokes, loops) in which the classes *are* nearly separable, which is the "learned features" insight of Chapter 8. The Runge function, by contrast, is a smooth function of one variable that a degree-8 polynomial already represents to the noise level; there was no feature left to learn. (2) "Of the 982 fours, 31 were classified as nines, while 12 of the 1009 nines were classified as fours" – the 31 lower the *recall* of class 4 (row), the 12 lower its *precision* (column). (3) The test set was used to choose hyperparameters, so 98.9% is an optimistic estimate – by how much nobody can say from these numbers, which is the problem: the one number that was supposed to be a prediction of performance on new data no longer is. Re-split, tune on a validation set, report the test number once. (4) Not quite: the Runge curves show the *mechanism* (Adam's insensitivity to γ, plain GD's stability limit), but item 6 asks about training *the* network, which includes MNIST. One hour: a γ scan over three decades for Adam and RMSprop and two batch sizes on a 10 000-image subset, test accuracy against epoch – the week-38 figure on the project's own data.

The last checks before Monday November 9

Reproducibility

- Restart the kernel, run everything top to bottom, and compare every number in the report with the output. Seeds fixed and stated.
- Every figure saved as PDF by the code (`plt.savefig`), named after what it shows, and produced by a script in the repository.
- Your tests, quoted: the gradient check of your backpropagation against automatic differentiation or central differences (as we did on Monday for the recurrent network); a linear network with no hidden layer reproducing OLS; agreement with Scikit-Learn or PyTorch on one configuration.
- Upload the notebook executed top to bottom, so that a reader sees the outputs.

Hand-in (Canvas, Monday November 9)

- One PDF, the report; the code as a link to your GitHub/GitLab repository with a folder of selected results and a README that says how to run it.
- Cite the data set (LeCun, Cortes and Burges, *The MNIST database of handwritten digits*) and the software you used, as the project text asks.
- The LLM appendix after the conclusions – missing means −5 points – consistent with your docstrings.
- References verified against the sources, in the course format.
- Group of two to three: all names on the report, one hand-in.

Wrap-up, the deadline, and Monday

- **No weekly exercises. Project 2 is due Monday November 9** – PDF on Canvas, code on GitHub, LLM appendix included.
- **If you want Monday's material in the project:** you do not have to. A group that treats the Runge regression as one-step forecasting along x and compares a recurrent network with its dense network, cross-validated the same way, is welcome to – as an addition, not a replacement.
- **Next Monday (week 46):** decision trees, ensemble methods and random forests (Chapter 7), then boosting – a non-linear model built without gradient descent, which will matter for Project 3.